

Mercans Interview — Master Answer Book

Senior Backend Developer (AI) | Satish Kumar | Prepared 13 August 2026
18 practised questions with best answers, scores, upgrade paths to 10/10 | Word-choice, pronunciation and grammar corrections at the end

Company name — say it right, every time: Mercans = MER-kanz. "MER" as in *mercy*, "kanz" as in *cans*. Stress the first syllable. Not "MNCs", not "Markans", not "McKinsey". Getting the company name wrong is the single fastest way to signal mass-applying.

Part 0 — How to Approach These Questions

0.1 The two question families

Family	What they are testing	Structure to use
Behavioural conflict, mistake, challenge, why leave, goals	Judgement, ownership, how you work with people, whether you learn from failure	STAR — Situation, Task, Action, Result. One story only. End on a number.
Technical / design scaling, consistency, API design, security	Whether you reason in patterns or in guesses; depth and vocabulary	Layered walk-through — name 3-6 layers up front, then fill each in one line.

0.2 STAR — the behavioural template

- **S — Situation (1 line):** where and when. "At Typo, a client asked for a custom feature..."
- **T — Task (1 line):** what *you* owned. "I owned the design decision."
- **A — Action (the bulk, 3-4 lines):** what **I** did — the decision, the trade-off, the judgement call.
- **R — Result (1 line, always a number):** "shipped a week early; 10-15 clients adopted it."

The line that lifts every behavioural answer to a 9: one explicit **judgement sentence** before the result — **"I pushed to pause and re-align before coding, even at the cost of a day, because shipping the wrong thing costs far more."** Interviewers score judgement, not activity.

0.3 Technical answers — structure first, then detail

1. **Announce the layers.** "I'd think about this in four layers — X, Y, Z, W." This alone reads as senior.
2. **One line per layer**, naming the **pattern by name** (Circuit Breaker, Saga, Outbox, Strangler Fig, CDC).
3. **Trade-off sentence.** "This gives eventual consistency, so I keep the window small and make writes idempotent."
4. **Land on the outcome.** Latency, throughput, cost, zero data loss, or a business metric.

0.4 Delivery rules — the ones you keep slipping on

Rule	Why / what it sounds like
Never restart the answer	When you lose your place, move to the next point . Restarting from the top is the single biggest score-killer — it makes a good answer sound unsure.
Say "I", not "you"	"I added a circuit breaker" — not "you add a circuit breaker". Ownership.
75-90 seconds	Compression was your biggest single improvement all session. Three crisp points beat nine loose ones.
Say the point once	You loop back to re-explain a point you already made. Say it, then stop.
Never call your own work "easy"	It deletes the difficulty you're being asked to demonstrate.
Land results positively	You once said "the client did complain" when you meant "no client complained about latency" . Phrase the win as a win.
Close cleanly	End with "That's my answer." Don't trail off into "...and yeah, these are the things."
Play your unique card	For an AI role: "I don't just use AI — I've built it in production." The Typo RAG pipeline. Never let an AI question end without it.

Part 1 — Behavioural Questions

Q1. Tell me about a time you disagreed with a teammate. Best: 8.5/10

BEST ANSWER (STAR)

- **S:** At Typo, a client asked for a custom feature that was specific to them.
- **T:** I saw the same need across other clients, so I proposed building it as a platform-wide capability instead of a one-off.
- **A:** My manager pushed back on bandwidth — a generic build would take longer. I didn't argue for my design; I looked for the option that satisfied both. I proposed a compromise: **deliver the client's requirement on the original timeline, but design the underlying module so it could be generalised later.** I aligned on that in one conversation and started building.
- **R:** We shipped ahead of schedule, and once generalised, **10-15 other clients adopted the same feature** — I confirmed adoption through our usage analytics.

TO REACH 10

- Add the **judgement line**: "I disagreed on scope, not on priority — so I looked for the design that met his constraint and mine."
- Name what you **gave up**: "I accepted a narrower launch scope in exchange for a reusable core." Naming the concession shows maturity.
- One line on the **relationship outcome**: "It also set the pattern for how we evaluate custom asks now."

Q2. What is the most challenging project you have worked on? Best: 8.5/10

BEST ANSWER

- **Lead with the architecture, not the story**: "The most challenging was building 'Ask AI' at Typo — a **RAG pipeline** over our product data."
- **The pipeline in one breath**: embed the user's query → match against the **vector database** → pass the retrieved context to the LLM so the answer is **grounded** in our own data → if confidence is low, **fall back to raising a support ticket** instead of guessing.
- **What made it hard**: (1) **grounding** — stopping the model from answering from general knowledge instead of our data; (2) **keeping embeddings fresh** as product data changed; (3) deciding the **fallback threshold** — when the system should admit it doesn't know.
- **R:** It reduced manual support effort by **40-50%**.

TO REACH 10

- Add **one measured detail**: chunking strategy, top-k retrieval, or how you evaluated answer quality. Any single concrete number ("top-5 chunks", "re-embedded nightly") proves you built it rather than read about it.
- Add the **failure you fixed**: one hallucination or stale-answer incident and what you changed. Challenge questions score highest when they contain a real setback.

Q3. Why are you leaving Typo, and why Mercans? Best: 8/10

BEST ANSWER

- **Positive about Typo**: "Four to five years at Typo, where I grew into leading a team of six and built our AI capability from scratch. Nothing negative — I've got a lot out of it."
- **The pull, not the push**: "I want to apply the same backend-plus-AI combination at a **larger, global scale**. Payroll is exactly that kind of domain — high data volume, multi-country, and security and compliance that genuinely matter."
- **Specific to Mercans**: "And it's a place where AI is being built *on top of* a serious platform, which is the work I want to do next."

TO REACH 10 (THIS IS YOUR WEAKEST-SCORING QUESTION — FIX IT)

- **Name one concrete Mercans fact**. Global payroll and EOR across many countries; clients like Accenture, Adidas, Tesla, Uber. One specific detail converts a generic answer into a researched one.
- **Name the domain challenge you find interesting** — **data residency and GDPR across jurisdictions**, or the correctness bar on money movement. That shows you've thought about *their* problem.
- **End forward-looking**: "I want my next four years to be building AI features on a platform where correctness is non-negotiable."

Q4. Tell me about a mistake you made in production. Best: 8.5/10

BEST ANSWER (STAR)

- **S:** We migrated a MySQL database from AWS to GCP with minimal downtime.

- **T:** I owned the delta-sync script that carried over changes made during the migration window.
- **A — the mistake, stated plainly:** "My delta script captured only **newly created** records. It missed records that **already existed and were updated** during the window — so some statuses in the new database were stale."
- **Detection:** caught it with a **timestamp-based regression check** comparing source and target.
- **Fix:** replaced the naive delta with **CDC — change data capture** — so both inserts and updates were streamed across.
- **R:** Data reconciled, and CDC became our standard approach for migrations after that.

TO REACH 10

- Add **impact and blast radius:** how many records, how long stale, whether a customer saw it. Owning the size of the mistake reads as more honest than describing it abstractly.
- Add the **systemic lesson**, not just the technical one: "The real lesson was that I validated the *happy path* of the migration but not the *mutation path* — now I write the reconciliation check **before** the migration script."

Q5. Where do you see yourself in five years? / What motivates you? Best: 8.5/10

BEST ANSWER

- **Depth:** "Going deeper on AI engineering — RAG, agents, and how to build reliably on top of models that keep changing."
- **Scope:** "Growing into a **staff or principal** level of technical ownership — architecture across systems, not just one service."
- **People:** "And continuing to grow as a leader. I lead six engineers today; in five years I'd like to be effective at that scale several times over."
- **Motivation:** "What actually motivates me is shipping something people use and seeing the number move — like the 40–50% support reduction from Ask AI."

TO REACH 10

- **Connect it to them:** "...and I'd want to do that growth *here* — payroll AI is a five-year problem, not a one-year one."
- **Pick a lane clearly.** Saying "staff/principal *and* management" can sound undecided. Better: "My centre of gravity is **technical** — staff-level architecture — with a team around me, which is exactly what I do today."

Q6. Tell me about a conflict across design, product and dev teams. Best: 8.5/10

BEST ANSWER (STAR)

- **S:** A custom client feature had been signed off by product and design and handed to engineering.
- **T:** While reviewing it before building, I found the requirements were **outdated** — they no longer matched what the client currently needed.
- **A:** Instead of escalating in writing, I called **one meeting with all three teams** and re-aligned on the current requirement. Then I fixed the cause, not just the incident: a **shared Slack channel** for the three teams, and a **beta-review gate** where design, product and engineering sign off together before development starts.
- **R:** It became our standard process; the client gave positive feedback, and we rolled the same gate out to other client projects.

TO REACH 10

- Add the **judgement line** — this is the one polish item still missing: "**I pushed to pause and re-align before writing code, even though it cost us a day, because shipping the wrong thing costs far more.**"
- Add one line on **handling the pushback:** someone always resists a pause. Say how you brought them around.

Q7. How do you break down a complex problem? Best: 8.5/10

BEST ANSWER

This scored well specifically because you **compressed it to a quarter of the original length**. Keep it that tight.

- **Frame:** "I break a large problem into independent, ownable pieces — and I do it for three reasons, only one of which is technical."
- **1 — Parallelism:** independent pieces can be built simultaneously by different engineers, which cuts delivery time.
- **2 — Ownership and isolated testing:** each piece has an owner and can be tested on its own, so bugs surface early rather than at integration.
- **3 — Distributed knowledge:** it spreads context across the team — better onboarding, and no single point of dependency on one person.
- **Plus fault isolation:** a failure in one piece doesn't take the whole feature down.
- **R:** "On our last few releases this got us to near-zero production bugs."

TO REACH 10

- **Anchor it in one real example** — walk through how you split one specific Typo feature into pieces and who owned

what. Abstract frameworks score 8; the same framework with one concrete decomposition scores 10.

- Add **how you decide the seams**: "I split along data ownership and along interfaces that are unlikely to change."

Part 2 — System Design & Distributed Systems

Q8. Your service suddenly gets 10x read traffic. How do you design for it? Best: 8.5/10

BEST ANSWER — LAYERED

- **Absorb at the edge:** **CDN** for static and cacheable content so most requests never reach the origin.
- **Scale horizontally:** **auto-scaling** behind a **load balancer**, triggered around 80–90% CPU; **consistent hashing** so adding nodes doesn't reshuffle every key.
- **Protect the database:** **Redis caching** for hot reads, plus **read replicas**; **connection pooling** so the DB isn't exhausted by connection count.
- **Shed and defer:** push non-critical work to **RabbitMQ** and process asynchronously; rate-limit abusive callers.
- **Stay available:** Redis **master-replica** for high availability.

TO REACH 10

- Add **cache invalidation strategy** — TTL vs write-through vs event-driven. This is the follow-up they will ask.
- Name the **thundering herd** / **cache stampede** risk and the fix (request coalescing, jittered TTLs). Naming that risk is a strong senior signal.
- Close with the **measure**: "I'd watch p99 latency and cache hit ratio to decide if it's working."

Q9. How do you stop one failing microservice cascading through the system? Best: 8/10

BEST ANSWER — FIVE NAMED PATTERNS

- **Circuit Breaker** (Resilience4j / Hystrix) — after N consecutive failures, stop calling the sick service and fail fast instead of piling up threads.
- **Timeouts** — never an unbounded wait; a slow dependency is more dangerous than a dead one.
- **Retries with exponential backoff and jitter** — jitter matters, or every client retries in sync and you re-create the spike.
- **Bulkhead** — separate thread pools / connection pools per dependency, so one saturated dependency can't consume all the capacity.
- **Fallback / graceful degradation** — serve cached or partial data rather than an error.

TO REACH 10

- Add the **failure story** in one line: "Without this, one slow dependency exhausts the caller's thread pool, and the caller's caller fails next — that's the cascade." Explaining the *mechanism* beats listing the cures.
- Add **observability**: "I'd alert on circuit-breaker open events and on p99 per dependency, so we see it before customers do."
- Mention **load shedding** as the last resort under extreme load.

Q10. How would you migrate a monolith to microservices — and roll back if it goes wrong? Best: 8.5/10

BEST ANSWER

- **Why, first:** independent deploys and independent scaling — but only if each service owns its data. **Database per service**; a shared database gives you a **distributed monolith**, the worst of both worlds.
- **How — Strangler Fig:** extract one bounded capability at a time, put an **API gateway / routing layer** in front, and shift traffic by percentage to the new service while the monolith still runs.
- **Data:** keep the two stores in sync during the transition with **CDC or dual-write**.
- **Rollback:** because the monolith is still live, rollback is a **routing change** — shift the traffic percentage back. No redeploy, no data restore.
- **Comms:** HTTP/REST externally, gRPC internally where latency matters.

TO REACH 10

- Say **how you choose the first service**: "Something with clear data boundaries and low coupling — usually not the core transaction path."
- Add the **dual-write hazard** and the fix — the **outbox pattern** (see Q11). Connecting the two topics unprompted is a 10-level move.

Q11. How do you keep data consistent across microservices? (a payroll run touches several services)

Best: 8.5/10

BEST ANSWER

- **Accept the constraint:** "Across services there is no distributed transaction — so I aim for **eventual consistency**, done safely."

- **Saga:** break the distributed transaction into a sequence of **local transactions**, one per service.
- **Compensating transactions:** if step 3 fails, you don't roll back — you **undo** steps 1 and 2 with explicit reversing operations.
- **Orchestration over choreography:** a central orchestrator drives the steps. Easier to reason about, debug and observe than services reacting to each other's events.
- **Outbox pattern** — solves the dual-write problem: in **one atomic database transaction**, write the business row *and* an event row into an outbox table. A background relay (CDC / Debezium) publishes the event to Kafka. So the data and the event can never disagree.
- **Close on the trade-off:** "That gives eventual consistency, so I keep the window small and make every write **idempotent**."

The concrete example to use: Satish pays Virat ₹70,000. The payroll service writes the payment row *and* the "PaymentInitiated" event in the same transaction. If the service crashes before publishing, the event is still safely in the outbox — the relay picks it up and publishes. Without the outbox, you either save the payment and lose the event, or publish an event for a payment that never saved.

TO REACH 10

- Say the **payroll-specific consequence** out loud: "In payroll, a lost event means someone doesn't get paid, or gets paid twice — so idempotency keys on the payment operation are non-negotiable."
- Add **reconciliation**: a periodic job comparing the ledger across services and flagging drift.

Correction to remember: eventual-consistency gaps are **not** fixed with table locks or unique constraints. They are managed by **processing steps fast** and making writes **idempotent**.

Q12. Your system must handle 1 million writes per minute. How do you keep it running?

Best: 9/10 — your highest score

BEST ANSWER

- **Set the scale:** "That's roughly **17,000 writes per second** — and unlike reads, you can't cache your way out of writes."
- **Buffer:** put **Kafka** (a durable log) in front — accept writes at the edge, let consumers drain at the database's pace.
- **Batch:** bulk-insert in batches (~10,000) rather than row-by-row — far fewer round trips.
- **Shard:** partition by a well-chosen **shard key** so writes spread evenly; a poor key creates **hot partitions**.
- **Right store:** a **write-optimised** store — Cassandra or DynamoDB (LSM-tree based) rather than a single relational primary.
- **Async acknowledgement** where the business allows it.

TO REACH 10

- Add **backpressure**: what happens when consumers fall behind — lag monitoring, autoscaling consumers, and a defined shed policy.
- Say which part is **not** negotiable for payroll: "For money, I'd keep the write path synchronous and durable even at the cost of throughput."

Q13. Follow-up: how do you guarantee not a single write is lost? Best: 9/10

BEST ANSWER — DURABILITY AT EVERY HOP

- **Producer:** `acks=all` plus retries — the producer only considers it written when all in-sync replicas confirm.
- **Broker:** **replication factor 3** — survive a broker loss.
- **Consumer:** **commit the offset only after the database write succeeds**. Committing first is how messages silently vanish.
- **Therefore:** you get **at-least-once delivery** — which means duplicates are possible.
- **So:** make every write **idempotent** — a unique business key with an upsert, so replaying the same message twice is harmless.

TO REACH 10

- Contrast **at-least-once vs exactly-once** in one line: "Exactly-once is expensive and narrow; at-least-once plus idempotency gets the same outcome more cheaply."
- Add the **dead letter queue** for messages that fail repeatedly, so a poison message doesn't block the partition.

Q14. How do you secure sensitive payroll data at scale? Best: 8/10

BEST ANSWER — THREE LAYERS PLUS COMPLIANCE

- **In transit:** TLS everywhere, including service-to-service.

- **At rest:** database encryption, plus **field-level encryption or tokenization** for the truly sensitive columns — bank account numbers, national ID, salary.
- **Access control:** **RBAC** with least privilege, **audit logs** on every read of sensitive data, and keys managed in a **KMS** — never in the codebase or environment files.
- **Compliance close:** "And because Mercans operates across many countries, **data residency and GDPR** shape the design — where data is stored matters as much as how."

Key correction: hashing is one-way. Use it for passwords and verification only. For data you must read back — salary, bank details — use **reversible encryption**. Do not say "I'd hash the salary."

TO REACH 10

- Add **key rotation** and **envelope encryption** (data key encrypted by a master key in KMS).
- Add **data minimisation and masking**: engineers see masked values in logs and lower environments by default.
- Add **secrets management** for the application itself (Vault / Secrets Manager, short-lived credentials).

Part 3 — API Design

Q15. How do you design an API? Walk me through versioning, communication style, ease of use and backward compatibility. **Best: 8.5/10**

BEST ANSWER — THE SIX LAYERS, IN THIS ORDER

1. **Contract first.** Get the requirement clear, then define the resources, the endpoints, the HTTP methods, and a **consistent request/response schema** across every endpoint.
2. **Communication style.** **REST** for client-facing, **gRPC** for internal service-to-service where latency and payload size matter, **GraphQL** where clients need flexible, query-heavy access.
3. **Versioning and backward compatibility.** Version in the URL (`/v1/`). **Additive changes only** within a version — add optional fields, never remove or rename. Retire old versions through a **deprecation policy**: announce, dual-support for a window, then sunset.
4. **Security.** Authentication and authorization, TLS/HTTPS, input validation. For a payroll API, **idempotency keys on write endpoints** so a retried request never pays someone twice.
5. **Reliability and scale.** **Rate limiting** so no client can flood the service; **caching for read-heavy endpoints** (Redis, or CDN for static content); **pagination or cursors** so responses stay small on the wire and the database isn't scanned unnecessarily.
6. **Documentation.** OpenAPI / Swagger, generated from the code.

Two corrections that cost you marks:

1. **Caching helps reads, not writes.** You said "if the API requires too many writes, add caching." Caching exists to avoid *re-reading*. Say: "for **read-heavy** endpoints I add caching."
2. **Public-facing does not mean "no authentication."** You said auth isn't required for public APIs — it's the opposite. Public/external APIs need **API keys or OAuth**. What you meant is that **internal** service-to-service calls may rely on network-level trust.

TO REACH 10

- Open by **announcing the layers**: "I'd think about it in six layers — contract, communication style, versioning, security, scale, documentation." Then fill each in a line.
- Say **idempotency keys** — for a payroll company this single line is worth more than any other sentence in the answer.
- Say **"additive changes only"** explicitly; that's the exact phrase they're listening for on backward compatibility.
- Use precise store vocabulary: **relational vs non-relational** (not "atomic DB"), and **static content** (not "starting content").

Deep-dive: the versioning mental model (asked and confirmed clear)

Purpose first: you version so **existing consumers keep working** while new consumers adopt the change. If ten client apps use v1 and you change v1's response, all ten break overnight. Instead you freeze v1, ship v2, and let clients migrate **on their own schedule**.

Change type	Examples	New version needed?
Additive (non-breaking)	Adding a new <i>optional</i> field to the response; adding a brand-new endpoint	No. Stays in the current version — existing clients ignore what they don't use.
Breaking	Removing a field; renaming a field; changing a field's type; making an input required	Yes. Ship it as <code>/v2</code> and keep <code>/v1</code> running.

The one-line rule: **add freely to the current version; create a new version only when you would remove, rename or change something that already exists.**

Maintenance nuance: rather than copy-pasting the v1 logic into v2, **share the common core and branch only where v2 genuinely differs** — otherwise every bug fix has to be applied twice. If asked "how do you maintain two versions", say: "shared core logic, with a thin version-specific layer on top."

Q16. When the API changes, how do you make sure the documentation stays updated? **Best: 8/10**

Relevance note: this almost never appears standalone — it's the natural **follow-up** to the API-design question. Keep it under a minute.

BEST ANSWER — THREE MECHANISMS, ONE LINE EACH

- **1. Generate docs from code — the spec is the source of truth.** OpenAPI/Swagger reads the annotations and method signatures, so the reference documentation can't drift from the implementation.

- **2. Gate it in CI/CD.** Any change to an endpoint must include the spec change **in the same pull request**; a CI check fails the build if the API changed and the spec file didn't. That converts documentation from goodwill into a requirement.
- **3. Contract testing.** Tools like Pact, or spec-validation tests, fail when the running API no longer matches the documented contract — proving the docs match **reality**, not just intent.
- **Plus one line:** "On top of the auto-generated reference, I use AI to draft human-friendly guides and examples for non-backend readers."

TO REACH 10

- Compress hard. Three mechanisms, one line each, then stop. This answer ran long and looped.
- Use the exact terms "**source of truth**", "**same PR**" and "**contract testing**" — each is a senior signal.
- Keep the AI-formatting point to **one sentence**. "Colours and pictures so people feel interested" reads junior; "human-friendly guides and examples on top of the generated reference" reads senior.

Part 4 — AI Judgement (critical for this role)

Q17. How do AI tools help you day to day? **Best: 8.5/10**

BEST ANSWER — THE FOUR-FINGER STRUCTURE

- **1. Tools:** name what you actually use day to day.
- **2. How they help:** "They take the boilerplate and the first draft off my plate — scaffolding, tests, repetitive transformations — so my time goes to **architecture and trade-offs**." **Never say "AI writes 80% of my code."**
- **3. I build AI, not just use it:** "I also build with it — I built the RAG pipeline behind Typo's Ask AI in production, so I understand what these systems can and can't do from the inside." **Never omit this for an AI role.**
- **4. But I verify:** "I treat output as a draft, not an answer — manual review plus end-to-end tests, especially on payroll and banking paths where AI has limited context across a large microservice codebase."

TO REACH 10

- Give **one specific example** of a task AI genuinely accelerated and by how much.
- Close with the framing sentence: **"AI is a multiplier I control — the human stays accountable."**

Q18. Where do you *not* trust AI? **Best: 8/10**

BEST ANSWER

- **Limited context in a large microservice estate:** "When five or six services interact, the model doesn't have the full picture — it fills the gap by guessing rather than solving. So I verify anything cross-service."
- **Secrets and credentials:** "I strip environment keys and credentials before giving a model access to a codebase. That access boundary is mine to control, not the tool's."
- **Critical money paths — banking and payroll:** "Especially destructive operations — deletes and updates against a production database. The blast radius is corrupted payroll data, so I want certainty, not probability."
- **Hallucination:** "Models state wrong things confidently — an invented library function or a non-existent API — so I verify against real documentation."
- **Outdated knowledge:** "There's a training cutoff, so for recent framework versions the answer can simply be stale."
- **Close positively:** **"So I treat AI as a powerful assistant I always verify. I trust it for boilerplate and drafts, and the human stays accountable for anything involving security, money or production data."**

TO REACH 10

- Name **hallucination** and **training cutoff** explicitly — interviewers listen for these two words.
- Don't end on "don't break your system" — that lands negative. End on the accountability line above.
- Say **"I'm cautious with AI"** rather than **"I doubt AI"** — measured, not dismissive.

Part 5 — Word Choice: Technical Terms You Used Incorrectly

Every one of these was said in a mock. Fix the word and the same answer scores higher — interviewers grade vocabulary precision as a proxy for depth.

You said	Say instead	Why it matters
"atomic DB"	relational / non-relational database	"Atomic" refers to the A in ACID, not a database category. Wrong word, wrong concept.
"starting content" (in a CDN)	static content	Static = unchanging assets. "Starting content" isn't a term.
"caching for writes"	caching for reads	The single most costly technical slip you made. Caches exist to avoid re-reading. Writes are handled with buffering (Kafka), batching and sharding.
"public API doesn't need authentication"	public/external APIs need API keys or OAuth; internal service-to-service may use network trust	You stated the security model backwards. An interviewer will catch this instantly.
"hash the salary"	encrypt (reversible) — hashing is one-way	Hashing is for passwords and verification. You can never read a hashed salary back.
"a regressive check"	a regression check	Regression = verifying nothing broke. "Regressive" means moving backwards.
"I doubt AI"	I'm cautious with AI / I don't rely on AI for...	"Doubt" reads dismissive; "cautious" reads senior and measured.
"I'm not reliable on AI"	I don't rely on AI	Your version says <i>you</i> are unreliable — it reverses the meaning entirely.
"move from v2 to v1 easily"	backward compatible through additive changes only	Names the principle rather than describing the symptom.
"lived" / "we lived the feature"	deployed / shipped / made it live	"Lived" isn't used this way in English.
"intimate / intimidate the user"	notify the user	Both alternatives mean something very different — this one can be genuinely awkward.
"parallelly"	in parallel	"Parallelly" is not standard English.
"very less bugs"	far fewer bugs	"Fewer" for countable things, "less" for uncountable.
"the client did complain" (meaning the opposite)	no client complained about latency	You accidentally reported your win as a failure. Always land the result positively.

Power vocabulary — deploy these deliberately

Each of these words, said once at the right moment, moves an answer up a point. Aim to use at least four in every technical answer.

Term	Term	Term
eventual consistency	CDC (change data capture)	idempotent / idempotency key
at-least-once delivery	offset (commit after write)	hot partitions
compensating transactions	orchestration vs choreography	outbox pattern
Strangler Fig	database per service	distributed monolith
circuit breaker / bulkhead	backoff with jitter	graceful degradation
additive changes	deprecation policy	contract testing
RBAC / least privilege	KMS / envelope encryption	data residency, GDPR
consistent hashing	read replica / connection pooling	p99 latency, cache hit ratio
source of truth	blast radius	backpressure / dead letter queue

Part 6 — Pronunciation Drills

Practise these out loud before the video screen. Mispronouncing a core term makes an interviewer question whether you've used it.

Word	You said	Say it like this
Mercans	MNCs / Markans / McKinsey	MER-kanz — "MER" as in mercy + "kanz" as in cans. Stress the first syllable. Most important item on this page.
Redis	readiness / radius / redes	RED-iss — like "red" + "iss". Your most stubborn one.
versioning	worsening	VER-zhun-ing — three beats. Central word for the API question.
HTTPS	STTPS	Spell it out: H-T-T-P-S .
writes	rights / ride	Enunciate the W : W-rites . Critical when contrasting reads and writes.
shard	short	SHARD — long "ar", ending in a hard D.
notify	intimidate / intimate	NO-ti-fy .
DB	"TV"	Say D-B crisply, or just say "the database".
CDN	—	C-D-N (fixed in Day 2 — keep it).
circuit breaker	—	SIR-kit BRAY-ker .
consistent hashing	—	HASH-ing , not "hashning".

Part 7 — Grammar Mistakes (Recurring)

These repeated across both days. Fixing the top three alone will noticeably raise how senior you sound.

You said	Correct
The top three — fix these first	
"So I take the requirement and I build..." (while narrating a past project)	Past tense when narrating: "I took the requirement and built ..." — your single most persistent slip.
"someone ask me", "he come to me"	Third-person -s: "someone asks me", "he comes to me".
"I said him", "I said to him the plan"	"I explained to him", "I told him".
Everything else, in order of frequency	
"the data is stale"	"the data are stale" — or simply rephrase: "the records were stale".
"it will be look like"	"it will look like".
"should also able to understand"	"should also be able to understand".
"these are very critical system"	"these are very critical systems " — plural noun after "these".
"in this areas"	"in these areas".
"too many request"	"too many requests ".
"secrets has been export"	"secrets are exposed ".
"so others and around who is reading"	"so others who read the API".
"when I working on..."	"when I am working on..." / "when I was working on...".
"it become the standard"	"it became the standard".
"more better"	" better " — never "more better".
"I am a leader of six team"	"I lead a team of six ".
"the DB don't have too much load"	"the database doesn't have too much load".
"one million times per second per minute"	Pick one unit and land it once: "a million requests per minute ".
"and yeah, these are the things which I can think of"	"That's my answer." — a clean close, every time.

Final checklist before the video screen

- Say **MER-kanz** out loud five times before you join.
- Behavioural → **STAR**, one story, one judgement line, end on a number.

- Technical → **announce the layers**, name the patterns, close on the trade-off and the metric.
- **Never restart** an answer. Lost your place? Move to the next point.
- Every AI question gets the line: **"I don't just use AI — I built a RAG pipeline in production."**
- Past tense when narrating. Third-person -s. Close with **"That's my answer."**